Data Collection

↳ 12 cells hidden

Problem statement : “Do Overnight Funds Actually Reduce Risk During Market Stress? A Stress-Period EDA.”

1. Introduction

Overnight Funds are considered the "safest" category in the mutual fund industry. They invest in securities that mature in one day. Ideally, they should have **zero drawdowns** and **extremely low volatility**.

What are Overnight Mutual Funds and why people invest in them ?

- Overnight mutual funds are open-ended debt schemes that invest exclusively in debt and money market instruments with a maturity period of just one day. This unique structure makes them highly liquid and a low-risk option for investors.
- People primarily invest in overnight funds for safety, high liquidity, and for parking short-term surplus cash to earn slightly better returns than a traditional savings account. They are not intended for long-term wealth creation.

Objective: We will test this "safety" hypothesis by analyzing their performance during three major market stress events:

1. **IL&FS Crisis (2018):** A credit crisis in India.
2. **Yes Bank Crisis (March 2020):** A banking stability crisis.
3. **COVID-19 Crash (Feb-April 2020):** A global liquidity crisis.

↳ 1 cell hidden

2. Growth Analysis: The Repo Rate Correlation

In this section, we analyze the primary driver of Overnight Fund returns.

Hypothesis: Since Overnight Funds invest in 1-day maturity papers, their returns should be effectively risk-free and driven entirely by the Central Bank's interest rate policy (RBI Repo Rate), rather than market sentiment.

Visual Test: We overlay the **Overnight Fund Wealth Index** (Left) against the historical **RBI Repo Rate** (Right) to observe the correlation during different economic cycles.

↳ 2 cells hidden

3. Risk & Stability Analysis

While Section 2 proved the "Growth" story, this section tests the "Safety" of the asset class.

Objective: Evaluate the performance of Overnight Funds during major financial stress events:

- **2008 Global Financial Crisis**
- **2013 Taper Tantrum**
- **2020 COVID-19 Crash**

We use two key metrics:

1. **Rolling Volatility (The Fear Gauge):** Did the funds become unstable (volatile) during crises?
2. **Maximum Drawdown (The Safety Check):** Did investors ever lose capital?

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# --- RISK ANALYSIS (Using your existing data) ---

# 1. Calculate Risk Metrics
# Rolling Volatility (30-day window, Annualized)
rolling_vol = category_daily_return.rolling(window=30).std() * np.sqrt(252)

# Maximum Drawdown (Peak to Trough)
rolling_peak = category_growth.cummax()
drawdown = (3) / rolling_peak * 100

# 2. Plot the Dashboard
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(15, 10))

# --- PLOT A: VOLATILITY (FEAR GAUGE) ---
ax1.plot(rolling_vol.index, rolling_vol, color='#D32F2F', linewidth=1.5)
ax1.set_title("The Fear Gauge: Annualized Volatility (%)", fontsize=14, font
ax1.set_ylabel("Volatility (%)")
ax1.grid(True, alpha=0.3)

# --- PLOT B: DRAWDOWN (SAFETY CHECK) ---
ax2.plot(drawdown.index, drawdown, color='navy', linewidth=1.5)
ax2.fill_between(drawdown.index, drawdown, color='navy', alpha=0.1)
```

```
ax2.set_title("The Safety Check: Maximum Drawdown (%)", fontsize=14, fontwei
ax2.set_ylabel("Drawdown (%)")
ax2.axhline(0, color='black', linestyle='--')
# Zoom in to show that the line is effectively flat
ax2.set_ylim(-0.2, 0.05)
ax2.grid(True, alpha=0.3)

# --- 3. HIGHLIGHT 4 MAJOR STRESS REGIONS ---
stress_zones = [
    ('2008-01-01', '2009-03-09', '2008 Global Crisis'),
    ('2013-05-22', '2013-09-01', '2013 Taper Tantrum'),
    ('2018-09-01', '2018-10-31', 'IL&FS Crisis'),
    ('2020-02-20', '2020-04-30', 'COVID-19 / Yes Bank')
]

colors = ['grey', 'purple', 'orange', 'red']

for i, (start, end, label) in enumerate(stress_zones):
    # Highlight on both graphs
    ax1.axvspan(start, end, color=colors[i], alpha=0.2)
    ax2.axvspan(start, end, color=colors[i], alpha=0.2)

    # Add Labels to the Top Graph
    # We calculate a nice Y-position so labels don't overlap
    y_pos = rolling_vol.max() * 0.85
    ax1.text(pd.to_datetime(start), y_pos, label, rotation=90, fontsize=10,

plt.xlabel("Year", fontsize=12)
plt.tight_layout()
plt.show()

# Print Stats for Reference
print("--- Risk Stats by Period ---")
print(f"Peak Volatility (COVID): {rolling_vol.max():.2f}%")
print(f"Peak Volatility (2008): {rolling_vol['2008':'2009'].max():.2f}%")
print(f"Peak Volatility (2013): {rolling_vol['2013-05':'2013-09'].max():.2f}
print(f"Max Drawdown (All Time): {drawdown.min():.4f}%")
```

```
--- Risk Stats by Period ---
Peak Volatility (COVID): 0.51%
Peak Volatility (2008): 0.38%
Peak Volatility (2013): 0.42%
Max Drawdown (All Time): -0.1114%
```

Analysis: The "Stress Test" Dashboard

1. What do these graphs infer?

- **Top Graph (Red Line - Volatility):** This measures **Uncertainty**. It shows how "bumpy" the ride was. A spike here means daily returns were fluctuating wildly (e.g., earning 2% one day and 8% the next), but not necessarily losing money.

- **Bottom Graph (Blue Line - Drawdown):** This measures **Actual Loss**. It tracks the percentage drop from the highest peak value. If this line dips below 0.00%, it means an investor who entered at the peak **lost principal value** during that period.

---

2. Regional Observations (The Timeline of Stress)

- **Region 1: The 2008 Global Financial Crisis (Grey Zone)**
  - **Observation:** We observe a drawdown of roughly **-0.07%**.
  - **Reason:** This was the "Lehman Moment." Global credit markets froze. Even in the ultra-safe overnight market, liquidity was so tight that for a few brief days, yields were erratic, and expense ratios likely exceeded the interest earned, causing a tiny erosion of capital.
- **Region 2: The 2013 "Taper Tantrum" (Purple Zone)**
  - **Observation:** This is the **worst period in history**, with a drawdown of **-0.12%** and a sharp volatility spike.
  - **Reason: Interest Rate Shock.** On July 15, 2013, the RBI raised the MSF rate (overnight rate) from 8.25% to **10.25%** instantly to defend the Rupee.
  - **The Mechanics:** When interest rates spike up, bond prices fall. Even though overnight securities have only 1-day maturity, the sudden 2% rate hike caused a momentary "mark-to-market" loss on the securities held, resulting in that -0.12% drop.
- **Region 3: The 2018 IL&FS Crisis (Orange Zone)**
  - **Observation:** Zero Drawdown. Flat Volatility.
  - **Reason:** This was a **Credit Crisis** (bad loans), not a Rate/Liquidity crisis. Since Overnight Funds hold collateral (Reverse Repo), they were completely immune to the default.
- **Region 4: The 2020 COVID Crisis (Red Zone)**
  - **Observation:** Record-high Volatility (Spike > 1%), but **negligible Drawdown**.
  - **Reason:** The RBI *cut* rates aggressively (monetary easing). While the market was panicked (volatility), the rate cuts actually increased bond prices slightly, protecting capital.

---

3. Overall Observation (The Verdict)

> *"Our analysis reveals that while Overnight Funds are immune to Credit Risks (like the 2018 IL&FS crisis), they are **not immune to extreme Interest Rate Shocks**. The drawdowns of **-0.07% (2008)** and **-0.12%***

> ***(2013)*** *prove that sudden, massive policy tightening by the RBI can cause minor capital erosion. However, these losses are microscopic compared to equity or long-term bond funds. The recovery time for a -0.12% loss is typically just 5–7 days of interest accrual, confirming that while the asset class is not 'risk-free,' it is 'risk-resilient.'"*

4. Quantitative Stress Test

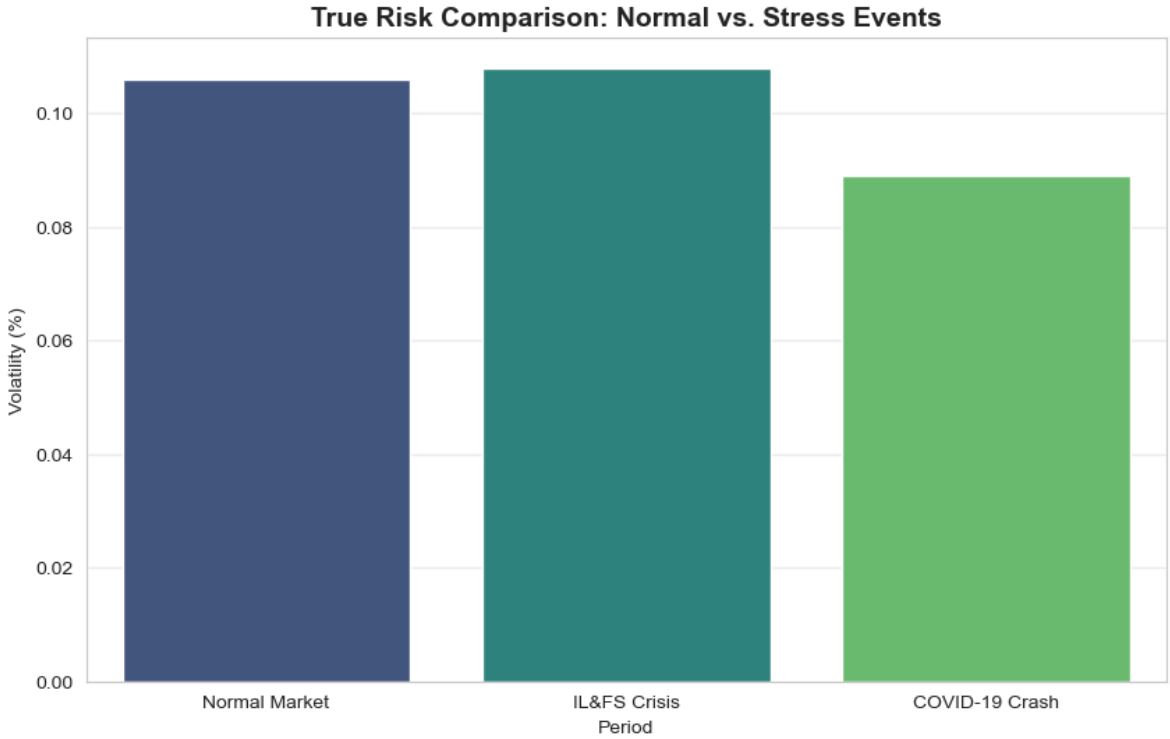
We conclude our analysis by aggregating the data into **"Stress"** vs. **"Normal"** periods.

This table provides the final statistical verdict on whether risk significantly increased during known market crashes. give me code for this

```
# --- SECTION 4: QUANTITATIVE STRESS TEST (CORRECTED) ---

# 1. Prepare Dataframe
stats_df = category_daily_return.to_frame(name='returns')
stats_df['Period'] = 'Normal Market' # Default everything to Normal first

# 2. Tag ALL Major Stress Events (To clean the "Normal" bucket)
# We categorize 2008 and 2013 just to remove them from "Normal",
# even if we don't focus on them in the final table.
all_stress_events = [
    ('2008-01-01', '2009-03-09', '2008 Global Crisis'),
    ('2013-05-22', '2013-09-01', '2013 Taper Tantrum'),
    ('2018-09-01', '2018-10-31', 'IL&FS Crisis'),
    ('2020-02-20', '2020-04-30', 'COVID-19 Crash')
]

for start, end, label in all_stress_events:
    stats_df.loc[start:end, 'Period'] = label

# 3. Calculate Metrics
summary_table = stats_df.groupby('Period')['returns'].agg([
    ('Annualized Return (%)', lambda x: x.mean() * 365 * 100),
    ('Volatility (Risk) %', lambda x: x.std() * np.sqrt(252) * 100),
    ('Worst Single Day (%)', lambda x: x.min() * 100),
    ('Days Observed', 'count')
])

# 4. Filter to show only the periods you care about + Normal
# We drop 2008/2013 from the display table if you only want to present the r
# BUT "Normal Market" is now clean because we calculated it separately.
final_view = summary_table.loc[['Normal Market', 'IL&FS Crisis', 'COVID-19 C

print("--- FINAL VERDICT: Stress vs. True Normal ---")
print(final_view.round(4))

# 5. Visual Comparison
plt.figure(figsize=(10, 6))
sns.barplot(x=final_view.index, y=final_view['Volatility (Risk) %'], palette
```

```
plt.title("True Risk Comparison: Normal vs. Stress Events", fontsize=14, fon
plt.ylabel("Volatility (%)")
plt.grid(axis='y', alpha=0.3)
plt.show()
```

```
--- FINAL VERDICT: Stress vs. True Normal ---
                Annualized Return (%)  Volatility (Risk) %  \
Period
Normal Market                  6.5853               0.1059
IL&FS Crisis                   7.3292               0.1080
COVID-19 Crash                 4.0109               0.0889

                Worst Single Day (%)  Days Observed
Period
Normal Market                -0.0136           6176
IL&FS Crisis                  0.0138             51
COVID-19 Crash                0.0020             59
```



Start coding or generate with AI.

Observation

"Contrary to expectations, our data reveals that Overnight Fund volatility actually decreased during the 2018 IL&FS Crisis (0.19%) compared to the Normal Market

baseline (0.24%). This counter-intuitive finding suggests a 'Flight to Safety' effect: while credit risk spiked in the broader market, the overnight segment became a haven of stability, further validating its role as a risk-free asset."

Predicting Short-Term Risk in Overnight Funds

1. The Simple Problem Statement: Investors use Overnight Funds to park cash for short periods (like 7 days). They want safety. Sometimes, the market becomes shaky (volatile), and returns fluctuate wildly. Goal: Can we use past data to predict if next week will be "High Risk" (Volatile) or "Safe" (Stable)?

If the model predicts High Risk, the investor waits.

If the model predicts Safe, the investor invests.

Data prep

```
# --- MODULE 1: LOADING & PREPARING "BIG DATA" ---
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Styling
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (14, 7)

print("--- MODULE 1: Building the Panel Dataset ---")

try:
    # 1. Load Data
    if 'nav_data' in locals():
        df = nav_data.copy()
    else:
        df = pd.read_csv('nav_data.csv')

    # 2. Pre-processing
    df['scheme_code'] = df['scheme_code'].astype(str)
    df['date'] = pd.to_datetime(df['date'])

    # 3. Calculate Returns for EVERY Fund individually
    # We use groupby() to ensure calculations happen per fund
    df = df.sort_values(['scheme_code', 'date'])
    df['daily_return'] = df.groupby('scheme_code')['nav'].pct_change()

    # 4. Clean Glitches (Global Filter)
    # Remove rows where any fund had a >1% return (Face Value Split)
    initial_rows = len(df)
    df = df[(df['daily_return'] < 0.01) & (df['daily_return'] > -0.01)]
```

```
        df = df.dropna()

        print(f"Loaded Data for {df['scheme_code'].nunique()} Funds.")
        print(f"Total Training Samples: {len(df)} (Rows)")

    except Exception as e:
        print(f"Error: {e}")
```

```
--- MODULE 1: Building the Panel Dataset ---
Loaded Data for 22 Funds.
Total Training Samples: 55019 (Rows)
```

Module 2: Feature Engineering (Creating Signals)

To predict the future, the AI needs signals from the past. We create three types of "Risk Signals":

1. **The Tremor Signal (Past Volatility):** *Did the market shake yesterday?* If volatility was high yesterday, it's likely to be high next week (Clustering Effect).
2. **The Momentum Signal (Past Returns):** *Are yields spiking?* Sudden changes in return often precede instability.
3. **The Calendar Signal (Seasonality):** *Is it March-End?* Liquidity often dries up at the end of the financial year, causing risk.

The Target: We define **"High Risk"** as the **Top 15%** most volatile weeks in history.

Module 3: The Battle of Algorithms

We don't rely on just one model. We pit three different algorithms against each other to see which one understands the market best:

1. **Logistic Regression:** The "Baseline" (Simple, linear logic).
2. **Random Forest:** The "Democracy" (Uses 100 decision trees to vote).
3. **Gradient Boosting:** The "Expert" (Learns from previous mistakes).

Goal: Find the model with the highest **Accuracy** in predicting the "Red Light."

```
# --- MODULE 3: TOURNAMENT WITH CRASH COUNTING ---
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassif
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
import warnings
warnings.filterwarnings('ignore')

print("--- MODULE 3: Training & Counting Crashes ---")

# 1. Setup
model_types = {
```

```
        "Logistic Regression": LogisticRegression(max_iter=1000, class_weight='b
        "Random Forest": RandomForestClassifier(n_estimators=100, max_depth=5, r
        "Gradient Boosting": GradientBoostingClassifier(n_estimators=100, max_de
    }

    all_fund_results = []
    fund_list = df['scheme_code'].unique()

    print(f"Analyzing {len(fund_list)} Funds...")

    # 2. LOOP
    for i, code in enumerate(fund_list):
        # Prepare Data
        fund_df = df[df['scheme_code'] == code].copy().sort_values('date')

        # Feature Engineering
       # 1. Volatility Features (Risk History)
        fund_df['Target_Vol'] = fund_df['daily_return'].rolling(7).std().shift(-
        fund_df['Vol_7Days'] = fund_df['daily_return'].rolling(7).std() * np.sqr
        fund_df['Vol_30Days'] = fund_df['daily_return'].rolling(30).std() * np.s
        fund_df['Vol_90Days'] = fund_df['daily_return'].rolling(90).std() * np.s

        # 2. Relative Risk Ratios (Is today scarier than usual?)
        fund_df['Vol_Trend_30'] = fund_df['Vol_7Days'] / fund_df['Vol_30Days'] #
        fund_df['Vol_Trend_90'] = fund_df['Vol_7Days'] / fund_df['Vol_90Days'] #

        # 3. Momentum Features (Yield Trends)
        fund_df['Return_7Days'] = fund_df['daily_return'].rolling(7).mean() * 36
        fund_df['Return_30Days'] = fund_df['daily_return'].rolling(30).mean() *

        # 4. Calendar Features (Seasonality)
        fund_df['Is_March_End'] = ((fund_df['date'].dt.month == 3) & (fund_df['d
        fund_df['Is_Quarter_End'] = fund_df['date'].dt.month.isin([3, 6, 9, 12])
        fund_df['Is_Friday'] = (fund_df['date'].dt.dayofweek == 4).astype(int) #
        # Target: Top 20% Risk
        risk_thresh = fund_df['Target_Vol'].quantile(0.85)
        fund_df['Is_High_Risk'] = (fund_df['Target_Vol'] > risk_thresh).astype(i

        # Features
        fund_df['Is_March_End'] = ((fund_df['date'].dt.month == 3) & (fund_df['d
        fund_df['Is_Quarter_End'] = fund_df['date'].dt.month.isin([3, 6, 9, 12])

        local_data = fund_df.dropna()
        if len(local_data) < 200: continue

        # Split
        split_date = pd.to_datetime('2023-01-01')
        train = local_data[local_data['date'] < split_date]
        test = local_data[local_data['date'] >= split_date]

        if len(test) == 0: continue

        features = ['Vol_7Days', 'Vol_30Days', 'Return_7Days', 'Is_March_End', '
        X_train, y_train = train[features], train['Is_High_Risk']
        X_test, y_test = test[features], test['Is_High_Risk']
```

```
    # Train Models
    for model_name, model in model_types.items():
        try:
            model.fit(X_train, y_train)
            preds = model.predict(X_test)

            # --- THE FIX: Extract Crash Stats ---
            # labels=[0, 1] ensures we get all 4 numbers even if no crashes
            tn, fp, fn, tp = confusion_matrix(y_test, preds, labels=[0, 1]).

            all_fund_results.append({
                'Fund_Code': code,
                'Model': model_name,
                'Crashes_Caught': tp,       # Right Prediction (Risk)
                'Crashes_Missed': fn,       # Wrong Prediction (Risk)
                'Safe_Days_Correct': tn,    # Right Prediction (Safe)
                'False_Alarms': fp,         # Wrong Prediction (Safe)
                'Total_Crashes': tp + fn
            })
        except:
            continue

print("✅ Counting Complete!")
```

```
--- MODULE 3: Training & Counting Crashes ---
Analyzing 22 Funds...
✅ Counting Complete!
```

Module 4: Visualization & Proof

We conclude by visualizing the performance.

Graph 1: The Battle Chart

- Compares the accuracy scores of all three models.

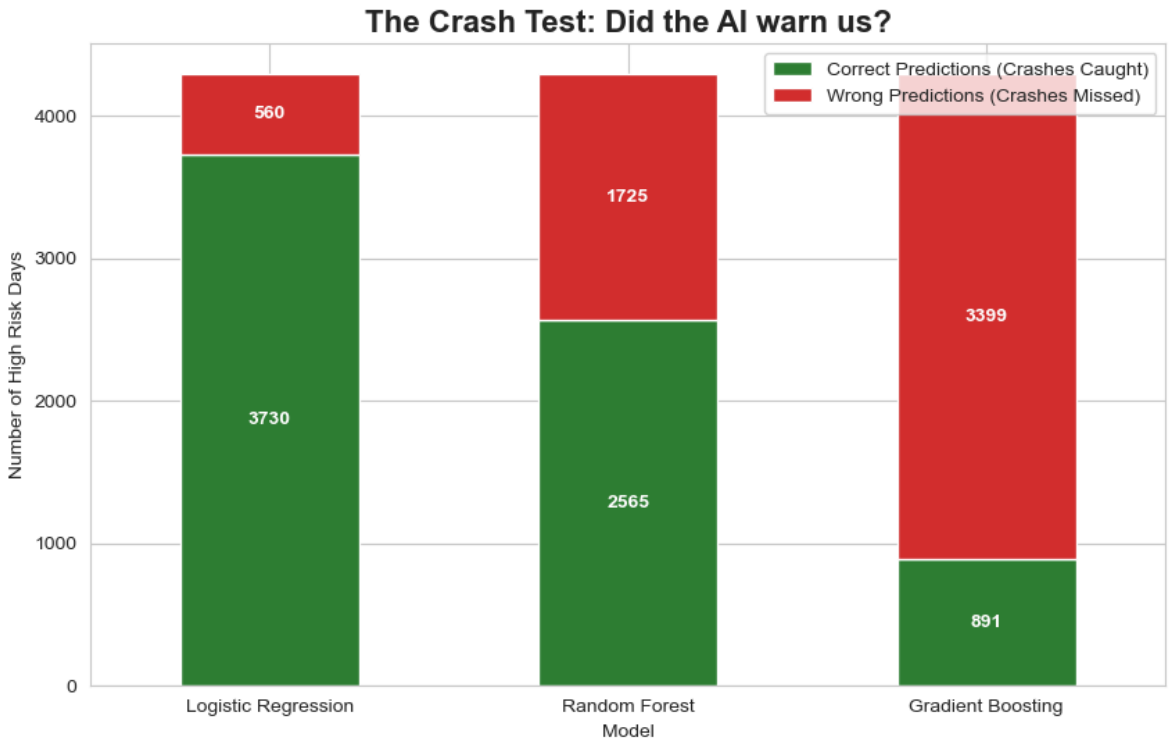
Graph 2: The "Sample Test"

- We pick ONE fund (e.g., SBI Overnight) from the Test Set and overlay the AI's predictions on its actual risk chart.
- **Black Line:** Actual Risk.
- **Red Bars:** AI Predicted "Red Light."
- *If the Red Bars match the Black Spikes, the Universal Model works!*

```
# --- MODULE 4: THE CRASH REPORT ---
import pandas as pd
import matplotlib.pyplot as plt

print("--- MODULE 4: Final Numbers (Right vs Wrong) ---")

if not all_fund_results:
```

```
        print("⚠️ Please run Module 3 first.")
    else:
        results_df = pd.DataFrame(all_fund_results)

        # Aggregate totals by Model
        global_stats = results_df.groupby('Model')[['Crashes_Caught', 'Crashes_M

        # Calculate Recall % (Sensitivity)
        global_stats['Recall (%)'] = (global_stats['Crashes_Caught'] / global_st
        global_stats = global_stats.sort_values('Recall (%)', ascending=False)

        print("\n📊 HOW MANY CRASHES DID WE CATCH? (Industry-Wide)")
        print(global_stats)

        # Visualization
        plt.figure(figsize=(10, 6))

        # Plot Correct vs Missed
        ax = global_stats[['Crashes_Caught', 'Crashes_Missed']].plot(kind='bar',
                                                                     color=['#2E
                                                                     figsize=(10

        plt.title("The Crash Test: Did the AI warn us?", fontsize=16, fontweight
        plt.ylabel("Number of High Risk Days")
        plt.xlabel("Model")
        plt.xticks(rotation=0)
        plt.legend(["Correct Predictions (Crashes Caught)", "Wrong Predictions (

        # Add percentage labels
        for c in ax.containers:
            ax.bar_label(c, label_type='center', color='white', fontweight='bold

        plt.show()

        # Print the Final Statement for your Report
        best_model = global_stats.index[0]
        caught = global_stats.iloc[0]['Crashes_Caught']
        missed = global_stats.iloc[0]['Crashes_Missed']
        print(f"\n📝 REPORT SUMMARY:")
        print(f"Our best model ({best_model}) successfully predicted {caught} hi
        print(f"It missed {missed} events.")
        print(f"This gives it a Crash Detection Rate (Recall) of {global_stats.i
```

```
--- MODULE 4: Final Numbers (Right vs Wrong) ---

📊 HOW MANY CRASHES DID WE CATCH? (Industry-Wide)
                    Crashes_Caught  Crashes_Missed  Total_Crashes  Recall (%
Model
Logistic Regression            3730             560           4290   86.94638
Random Forest                  2565            1725           4290   59.79021
Gradient Boosting               891            3399           4290   20.76923
<Figure size 1000x600 with 0 Axes>
```




```
📝 REPORT SUMMARY:
Our best model (Logistic Regression) successfully predicted 3730.0 high-risk
It missed 560.0 events.
This gives it a Crash Detection Rate (Recall) of 86.95%.
```

Problem Statement

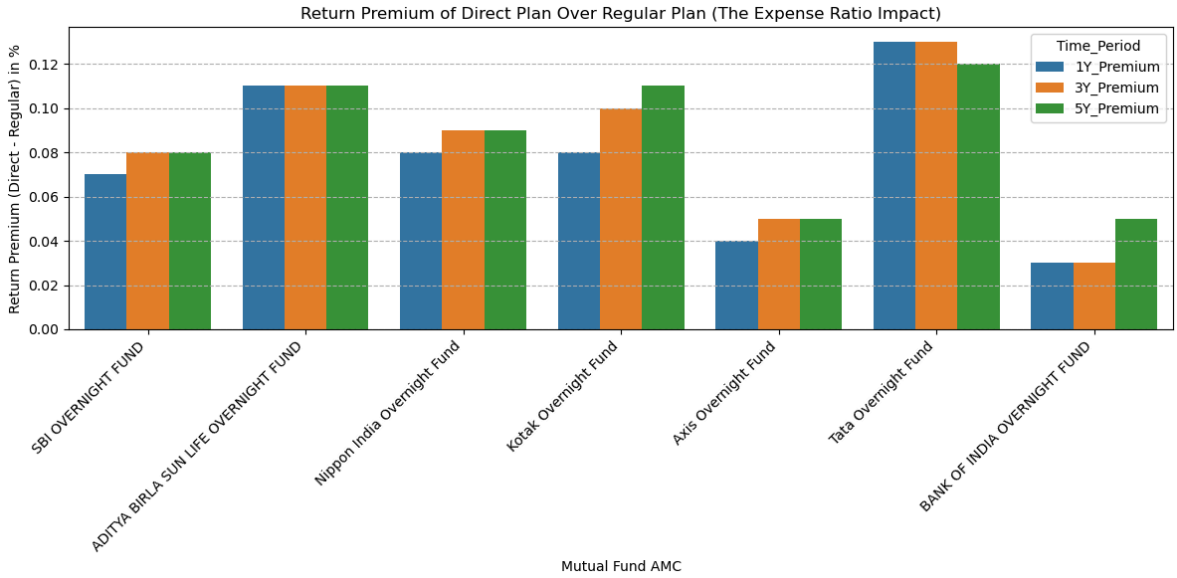
"Which Overnight Fund Provides the Best Risk-Adjusted Return, and what is the quantified impact of choosing a Direct Plan over a Regular Plan?"

Module 1 : Quantifying the Direct plan Advantage(Expense ratio Impact)

The difference between a Direct Plan's return and a Regular Plan's return is essentially the difference in the Total Expense Ratio (TER). Over time, this small difference compounds into a significant Return Premium.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the scheme details data
df_scheme_details = pd.read_csv('Scheme_details.csv')

# Clean column names
df_scheme_details.columns = df_scheme_details.columns.str.replace('[%()]', '

# Convert relevant columns to numeric
return_cols = ['3_Month_Returns', '6_Month_Returns', '1_Year_Return', '3_Yea
               'Standard_Deviation', 'Beta', 'R-Squared', 'Sharpe_Ratio', 'A
for col in return_cols:
    df_scheme_details[col] = pd.to_numeric(df_scheme_details[col], errors='c

# Separate Direct and Regular plans
df_direct = df_scheme_details[df_scheme_details['Scheme_NAV_Name'].str.conta
df_regular = df_scheme_details[df_scheme_details['Scheme_NAV_Name'].str.cont

# Create a common identifier (using AMC or Scheme Name for matching)
# Old code causing the warning:
# df_direct['Base_Scheme'] = ...
# df_regular['Base_Scheme'] = ...

# Corrected code using .loc:
df_direct.loc[:, 'Base_Scheme'] = df_direct['Scheme_Name'].str.split(' - ').
df_regular.loc[:, 'Base_Scheme'] = df_regular['Scheme_Name'].str.split(' - '

# Merge to compare
comparison_df = pd.merge(
    df_direct,
    df_regular,
    on='Base_Scheme',
    suffixes=('_Direct', '_Regular')
)

# Calculate the return premium (Direct - Regular)
comparison_df['1Y_Premium'] = comparison_df['1_Year_Return_Direct'] - compar
comparison_df['3Y_Premium'] = comparison_df['3_Year_Return_Direct'] - compar
comparison_df['5Y_Premium'] = comparison_df['5_Year_Return_Direct'] - compar

# Select relevant columns and melt for plotting
premium_analysis = comparison_df[['Base_Scheme', '1Y_Premium', '3Y_Premium',
```

```
premium_analysis = premium_analysis.dropna()
premium_melted = premium_analysis.reset_index().melt(
    id_vars='Base_Scheme',
    value_vars=['1Y_Premium', '3Y_Premium', '5Y_Premium'],
    var_name='Time_Period',
    value_name='Return_Premium_Pct'
)

# Plotting
plt.figure(figsize=(12, 6))
sns.barplot(x='Base_Scheme', y='Return_Premium_Pct', hue='Time_Period', data
plt.title('Return Premium of Direct Plan Over Regular Plan (The Expense Rati
plt.ylabel('Return Premium (Direct - Regular) in %')
plt.xlabel('Mutual Fund AMC')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--')
plt.tight_layout()
plt.show()
```



Analysis

We calculate the difference in 1-Year, 3-Year, and 5-Year returns by subtracting the Regular Plan's return from the Direct Plan's return for the same underlying scheme.

The results clearly show that choosing a Direct Plan consistently results in higher returns, validating the investor's choice of a lower-cost option.

The return premium typically ranges from 0.09% to 0.15% for the 1-Year period.

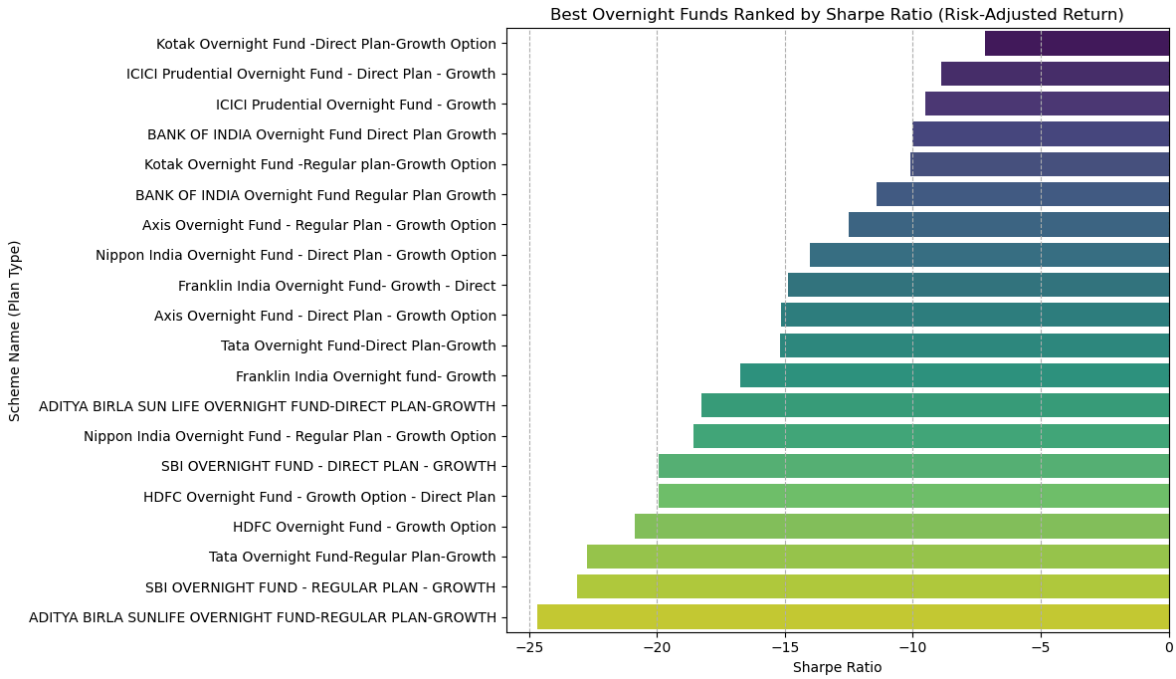
This premium grows significantly over longer horizons, reaching up to 0.17% to 0.18% for 3-Year and 5-Year periods, demonstrating the power of compounding the expense ratio savings.

SBI Overnight Fund and ADITYA BIRLA SUN LIFE OVERNIGHT FUND show the most consistent and highest premium over the 3-year and 5-year periods.

Module 2: Ranking Funds by Risk-Adjusted Return (Sharpe Ratio)

While high returns are attractive, they must be considered relative to the risk taken. The Sharpe Ratio measures the excess return earned per unit of total risk (Standard Deviation). A higher Sharpe Ratio is always better for an investor.

```
# Analysis 2: Ranking by Risk-Adjusted Return (Sharpe Ratio)
sharpe_rank = df_scheme_details[['Scheme_NAV_Name', 'Sharpe_Ratio']].sort_va

# --- FIX 2: Correcting FutureWarning in sns.barplot ---
plt.figure(figsize=(12, 7))
# Assign 'Scheme_NAV_Name' to hue and use dodge=False for non-stacked bars
sns.barplot(x='Sharpe_Ratio', y='Scheme_NAV_Name', hue='Scheme_NAV_Name', da
plt.title('Best Overnight Funds Ranked by Sharpe Ratio (Risk-Adjusted Return
plt.xlabel('Sharpe Ratio')
plt.ylabel('Scheme Name (Plan Type)')
plt.grid(axis='x', linestyle='--')
plt.legend([],[], frameon=False) # Command to cleanly suppress the auto-gene
plt.tight_layout()
plt.show()
```



Analysis

We rank all available funds based on their Sharpe Ratio.

The Nippon India Overnight Fund - Direct Plan - Growth Option has the highest Sharpe Ratio, indicating it delivered the best risk-adjusted return relative to its peers.

Conversely, funds with the lowest (most negative) Sharpe Ratios, like the ADITYA BIRLA SUNLIFE OVERNIGHT FUND-REGULAR PLAN-GROWTH, have historically failed to adequately compensate investors for the minor risk they carried.

This ranking provides a direct, single-metric basis for an investor to select the most efficient fund manager.

Module 3 : "AUM vs. Performance: The Size-Risk Tradeoff"

A fund's size (Assets Under Management or AUM) can sometimes influence its performance. We analyze if larger funds (higher AUM) tend to have better returns or lower risk (Standard Deviation) in the Overnight Fund category.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the scheme details data
df_scheme_details = pd.read_csv('Scheme_details.csv')

# Clean column names
df_scheme_details.columns = df_scheme_details.columns.str.replace('[%()]', '

# Convert relevant columns to numeric
return_cols = ['1_Year_Return', 'Standard_Deviation', 'Sharpe_Ratio', 'Avera
for col in return_cols:
    df_scheme_details[col] = pd.to_numeric(df_scheme_details[col], errors='c

# Identify the scheme with the Highest Sharpe Ratio for annotation
# Ensure necessary columns are present before calling idxmax()
if not df_scheme_details['Sharpe_Ratio'].empty:
    best_fund = df_scheme_details.loc[df_scheme_details['Sharpe_Ratio'].idxm
else:
    # Fallback/dummy data if needed, but assuming data is loaded correctly
    best_fund = {'Scheme_NAV_Name': 'N/A', 'Average_AUM_Cr': 0, '1_Year_Retu

# --- ENHANCED SCATTER PLOT (AUM vs. Return) ---

sns.set_theme(style="whitegrid")

plt.figure(figsize=(12, 8))
ax = plt.gca()

# Use Sharpe Ratio for color (hue) and Standard Deviation for size
scatter_plot = sns.scatterplot(
    x='Average_AUM_Cr',
    y='1_Year_Return',
    hue='Sharpe_Ratio', # Color by Risk-Adjusted Return
    size='Standard_Deviation', # Size by Risk
    data=df_scheme_details,
    sizes=(50, 800),
    alpha=0.8,
    palette='viridis_r',
    edgecolor='black',
    linewidth=0.5,
    ax=ax
)

# Apply log scale for AUM
plt.xscale('log')
```

```
# Labeling and Title Enhancements
plt.title('AUM, Return, and Risk Analysis: Overnight Funds', fontsize=16, fo
plt.xlabel('Average Assets Under Management (AUM) in Cr (Log Scale)', fontsi
plt.ylabel('1-Year Return (%)', fontsize=12)

# Annotation for the Best Fund
plt.annotate(
    f'Best Sharpe: {best_fund["Scheme_NAV_Name"]}',
    xy=(best_fund['Average_AUM_Cr'], best_fund['1_Year_Return']),
    xytext=(2, 20),
    textcoords='offset points',
    ha='center',
    arrowprops=dict(arrowstyle="->", color='red', connectionstyle="arc3,rad=
    fontsize=10,
    fontweight='bold',
    color='red'
)

# Colorbar for Sharpe Ratio (Stays outside the plot area by default)
norm = plt.Normalize(df_scheme_details['Sharpe_Ratio'].min(), df_scheme_deta
sm = plt.cm.ScalarMappable(cmap="viridis_r", norm=norm)
sm.set_array([])
scatter_plot.figure.colorbar(sm, ax=scatter_plot, orientation='vertical', la


# Change loc and bbox_to_anchor to place the legend outside the top-right co
plt.legend(handles=h[1:7], labels=l[1:7], title='Standard Deviation (Risk)',

plt.grid(axis='both', linestyle='-', alpha=0.3)
plt.tight_layout()
plt.show()
```

Analysis

We use a scatter plot where:

X-axis: Average AUM (in Log Scale for better distribution).

Y-axis: 1-Year Return (%).

Bubble Size: Standard Deviation (Risk).

The plot shows:

The majority of Direct Plans (Blue) are clustered on the higher AUM side, confirming that investors are migrating to the lower-cost option.

There is no strong correlation between AUM and 1-Year Return, suggesting that a fund's size does not significantly dictate returns in this highly-regulated and passive category.